Assignment: PCA, LDA, and SVD for Face Image Analysis and Movie Recommendation

Datasets

Q1 and Q2 use the PIE face dataset (Dataset 1), Q3 uses 2–3 images of your choice, and Q4 uses the MovieLens 100K dataset (Dataset 2).

Dataset 1: PIE Face Dataset (Q1 and Q2)

The PIE face dataset is provided in the folder `PCA_PIE_Data` (inside `assignment_01.zip`).

- The dataset has two folders: `lights` (room light on + flash, so shadows are soft) and `nolights` (room light off, only the flash, so shadows are dark).
- Both folders contain the 5 persons (`Person1`, `Person2`, `Person9`, `Person19`, `Person46`), with 21 photos per person.
- The photos are named `flash_01.jpg` to `flash_21.jpg`. The flash number gives the direction of the light: the same number means light from the same direction, for every person and in both folders.
- All photos are grayscale, of size 100×100.

Dataset 2: MovieLens 100K (Q4)

Use the publicly available MovieLens 100K dataset from GroupLens.
Dataset link: `https://grouplens.org/datasets/movielens/100k/`

Parameter	Description
Number of users	943
Number of movies	1,682
Number of ratings	100,000
Rating scale	1 to 5

Q1. Principal Component Analysis (PCA) on PIE Face Dataset

1. **Dataset Preparation**
   - The images are already grayscale and of size 100×100.
   - Flatten each image into a 1D vector and stack the vectors to form a matrix. Make one matrix for `lights` and one for `nolights` (105×10000 each).
2. **Apply PCA**
   - Compute PCA separately for the `lights` and the `nolights` data.
   - Reconstruct two selected images using Principal Components 1–3 (PC1–PC3) and visualize the results.
   - Repeat the reconstruction using Principal Components 5–9 (PC5–PC9) for the same images.

3. **Analysis**

   - Compare the reconstructed images from both component ranges.
   - Discuss:
     - Which set captures more visual and structural details?
     - How do reconstruction clarity and facial features change as more components are used?
     - Do PC1–PC3 show the person's face, or mostly the light and shadow? Is this different for `lights` and `nolights`?

Note: Do not limit your analysis to only two images. Apply the same PCA reconstruction process to multiple pairs of images (3–4 pairs), for example the same person with two different flash numbers, and two different persons with the same flash number. Observe whether the reconstruction quality and component behaviour remain consistent.

Q2. Linear Discriminant Analysis (LDA) for Face Class Separation

1. **Dataset Preparation**

   - Choose the photos of at least three persons (all 21 photos of each person).
   - Flatten each image into a 1D vector to form the input matrix, and create a label vector (the person number).

2. **Apply LDA**

   - There are more pixels (10000) than images, so first reduce the data with PCA (for example, to 20–40 components) and then apply LDA. Explain in one line why this step is needed.
   - Project the images into the LDA space and visualize the projection as a scatter plot (LD1 vs LD2), coloured by person.
   - Make the same scatter plot using PC1 vs PC2, for comparison.
   - Do this for both `lights` and `nolights`.

3. **Analysis**

   - Interpret the separation achieved by LDA.
   - Discuss:
     - How does LDA differ from PCA in its goal and results?
     - Which method seems better suited for distinguishing between persons' faces?
     - In the PCA plot, are the points grouped by person or by flash number? What about the LDA plot?

Note: Do not restrict your analysis to only one group of persons. Perform LDA on 3–4 different combinations of persons (for example, persons with and without glasses) to observe how the discriminant space and class separability vary.

Q3. Singular Value Decomposition (SVD) for Image Denoising

1. **Add Noise**
   - Take any 2–3 images of your choice. They need not be from the provided PIE data.
   - Add Gaussian or salt-and-pepper noise to each image. You may use an inbuilt function for this (e.g., `imnoise` in MATLAB or `skimage.util.random_noise` in Python).

2. **Apply SVD**
   - Decompose each noisy image using SVD: $A = U\Sigma V^T$.
   - Reconstruct each image using a limited number of singular values (top 5, 20 and 50).
   - Plot the singular values of the clean image and of the noisy image on the same graph.

3. **Denoising Analysis**
   - Compare the reconstructed images for different numbers of singular values.
   - Discuss:
     - How does SVD help in reducing noise? Use your singular-value plot to explain.
     - What trade-off do you observe between smoothness and image detail?
     - Which of your images needs more singular values to look good, and why?

Note: Extend your experiment by applying different types and levels of noise (e.g., Gaussian with varying variance, or salt-and-pepper with different densities). Observe how the effectiveness of SVD-based denoising changes with the type and strength of noise.

Q4. SVD-based Movie Recommendation System

A movie-streaming platform wants to recommend movies to users based on their previous ratings. The platform has collected ratings from many users for different movies. However, each user has rated only a small fraction of the available movies, resulting in a sparse user–movie rating matrix. The objective of this question is to develop an SVD-based recommendation system that estimates ratings for movies a user has not rated and recommends the highest-rated movies. Use the MovieLens 100K dataset (Dataset 2).

Students must implement the matrix construction, low-rank approximation, rating prediction, and recommendation steps. Do not use a prebuilt recommendation-system package that performs matrix factorization automatically.

1. **Construct the User–Movie Rating Matrix**
   Construct a matrix R in which each row corresponds to a user and each column corresponds to a movie. The entry $R(i, j)$ represents the rating given by user i to movie j. Use zero to represent an unrated user–movie pair. Required analysis (use all ratings for this step):
   - Determine the percentage of missing entries in the rating matrix.
   - Compute the average rating given by each user, using only the movies that user has rated (do not count the zeros).
   - Compute the average rating received by each movie, using only its observed ratings.
   - Identify the five most popular movies based on the number of ratings.

2. **Train/Test Split**
   Split the observed ratings into a training set (80%) and a test set (20%) using a fixed random seed. Build the training matrix R_{train} in the same way as R, but using only the training ratings, so that test pairs are zero in R_{train}. Steps 3–6 use R_{train} only.

3. **Perform Singular Value Decomposition**
   Compute the singular value decomposition of the training rating matrix:

   $$R_{\text{train}} = U\Sigma V^T$$

   Here, U contains user-side singular vectors, V contains movie-side singular vectors, and Σ is the diagonal matrix of singular values. Retain the first k singular values and corresponding singular vectors for $k \in \{5, 10, 20, 50, 100\}$:

   $$R_k = U_k \Sigma_k V_k^T$$

   Answer the following:

   - Plot the singular values in descending order.
   - Plot the cumulative fraction of squared singular values (energy) captured as k increases.
   - Determine the minimum k required to retain at least 90% of the total squared-singular-value energy.

4. **Predict Missing Movie Ratings**
   Use the rank-k approximation to estimate the rating of user i for movie j. The predicted rating is:

   $$\hat{R}_{ij} = \sum_{l=1}^{k} U_{il}\, \sigma_l\, V_{jl}$$

   Compute predictions for user–movie pairs that were not observed in the training data. For display as recommendations, clip predicted values to the rating scale $[1, 5]$. Use the raw (unclipped) predictions when computing MAE and RMSE in step 6.

5. **Generate Movie Recommendations**
   Select five users (for example, randomly with a fixed random seed). For each selected user:

   (a) Find all movies that the user has not rated (in either the training or the test set).
   (b) Predict the ratings for these movies using the rank-k model.
   (c) Sort the unrated movies in descending order of predicted rating.
   (d) Recommend the top 10 movies and display their titles and predicted ratings.

   Use a table such as the following in your report:

Movie title	**Predicted rating**
Movie A	4.82
Movie B	4.71
Movie C	4.65

6. **Evaluate the Recommendation System**
   Use the 80:20 train/test split from step 2. Perform all model fitting using the training set only. Evaluate predictions only on held-out ratings in the test set.

   Calculate the Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{N} \sum_{(i,j)\in\Omega} \left| R_{ij} - \hat{R}_{ij} \right|$$

   Calculate the Root Mean Square Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{(i,j)\in\Omega} \left(R_{ij} - \hat{R}_{ij} \right)^2}$$

   Here, Ω denotes the set of user–movie pairs in the test set and N is the number of test ratings. Compare MAE and RMSE for each value of k, and discuss how filling unrated pairs with zero affects the predicted ratings and the test error. Ensure that test ratings are not used when constructing the training matrix or computing any quantities used to fit the model.

Required plots and outputs for Q4:

- Heatmap or sparsity visualization of the user–movie rating matrix.
- Plot of singular values.
- Cumulative energy plot based on squared singular values.
- Plot of test-set RMSE versus k (and optionally MAE versus k).
- Top-10 recommendations for each of the five selected users.

Implementation notes for Q4:

- Use a fixed random seed for the train/test split so results are reproducible.
- Do not include test ratings in the matrix used to compute SVD.
- When calculating averages, use only observed ratings; never count the zeros that mark unrated pairs. Any average used for prediction, such as the fallback below, must be computed from the training set only.
- A conventional SVD of a zero-filled matrix is a baseline, not a complete solution to the missing-data problem; in this assignment, it is the required method. More advanced methods optimize only over observed ratings.
- For users with no ratings in the training set, or movies with no training ratings, a fallback such as the global training-set mean may be needed. State how such cases are handled.

Submission Instructions

- Do not use inbuilt functions that directly perform PCA, LDA or SVD, such as `pca`, `svd`, `svds` or `fitcdiscr` in MATLAB, or `numpy.linalg.svd`, `scipy.sparse.linalg.svds` and scikit-learn's `PCA`, `LinearDiscriminantAnalysis` or `TruncatedSVD` in Python. Implement these methods yourself. The only inbuilt function allowed for them is one that computes eigenvalues and eigenvectors (e.g., `eig` in MATLAB, `numpy.linalg.eig` in Python).
- For supporting steps, such as reading images or files, adding noise, sorting and plotting, inbuilt functions are allowed.
- **You must submit two separate files:**
  1. **Code Notebook (.m / .ipynb):** Containing only your implementation. Do not include any observations or analysis in the notebook.
  2. **Report (.pdf or .docx):** Include results, visual outputs, and detailed discussions for each question.
- Your report should clearly include:
  - Image results (original, noisy, reconstructed, projected)
  - Observations and analysis for each task
  - Discussion of findings and insights
  - Analysis by changing different choices of parameters